DRAFT - MF for hierarchial data

Center for Veterinary Biologics - Statistics Section

November 9, 2018

Contents

INTRODUCTION **1**

1 Quick Start **3**
Calculated MF. 3
Bootstrapped MF. 5

2 Reproducibility from rank tables **8**
Calculated MF. 8
Bootstrapped MF. 10

3 Calculating the rank table directly **14**
Calculated 14
Bootstrapped 14

Appendix I **17**

Appendix II **17**

INTRODUCTION

In the specific instance of experimental data that has been organized into a hierarchical structure, that is, with two levels of treatment (e.g. control and vaccinate) and nested grouping levels (e.g. multiple litters within a single pen, multiple pens within a single room), it is not appropriate to use functions `MFClus` or `MFClusBoot` to calculate the mitigated fraction without advanced knowledge of how to filter and reshape the input data.

The new functions of `MFh`, `MFnest`, `MFClusHier`, `MFhBoot`, `MFnestBoot`, and `MFClusBootHier` allow for evaluation of the mitigated fraction dataset without the need to manipulate the input data.

Examples

This tutorial uses an example dataset of experimental values of the fraction of lungs with gross leisions and additional housing condition information. A graphical display of the experimental design is also shown in Appendix I.

Example calculations include how to calculate the mitigated fraction at different variable levels, with and without bootstrapping and how to access returned values for reproducibility.

Throughout this document, we use the term *core variable* to indicate the variable corresponding to the lowest node of the hierarchical tree. *Nest variables* are those variables above the core. *All* refers to the entire tree structure.

```r
require(tidyverse)
require(MF)

## example data
a <- data.frame(
 room = paste('Room', rep(c('W', 'Z'), each = 24)),
 pen = paste('Pen', rep(LETTERS[1:6], each = 8)),
 litter = paste('Litter', rep(11:22, each = 4)),
 tx = rep(rep(c('vac', 'con'), each = 2), 12),
 stringsAsFactors = FALSE
 )
set.seed(76153)
a$lung[a$tx == 'vac'] <- rnorm(24, 5, 1.3)
a$lung[a$tx == 'con'] <- rnorm(24, 7, 1.3)
a <- a[-48,]
a
```

```
##      room   pen    litter  tx     lung
## 1  Room W Pen A Litter 11 vac 5.633023
## 2  Room W Pen A Litter 11 vac 4.618143
## 3  Room W Pen A Litter 11 con 9.196909
## 4  Room W Pen A Litter 11 con 7.277479
## 5  Room W Pen A Litter 12 vac 3.760431
## 6  Room W Pen A Litter 12 vac 3.856939
## 7  Room W Pen A Litter 12 con 6.307358
## 8  Room W Pen A Litter 12 con 3.521230
## 9  Room W Pen B Litter 13 vac 4.356031
## 10 Room W Pen B Litter 13 vac 6.098324
## 11 Room W Pen B Litter 13 con 7.866934
## 12 Room W Pen B Litter 13 con 8.339948
## 13 Room W Pen B Litter 14 vac 5.389661
## 14 Room W Pen B Litter 14 vac 5.799638
## 15 Room W Pen B Litter 14 con 8.275256
## 16 Room W Pen B Litter 14 con 7.952039
## 17 Room W Pen C Litter 15 vac 4.855498
## 18 Room W Pen C Litter 15 vac 5.657725
## 19 Room W Pen C Litter 15 con 7.658824
## 20 Room W Pen C Litter 15 con 8.517004
## 21 Room W Pen C Litter 16 vac 4.169034
```

```
## 22 Room W Pen C Litter 16 vac 4.837650
## 23 Room W Pen C Litter 16 con 5.822156
## 24 Room W Pen C Litter 16 con 7.718757
## 25 Room Z Pen D Litter 17 vac 3.595465
## 26 Room Z Pen D Litter 17 vac 4.921762
## 27 Room Z Pen D Litter 17 con 5.222559
## 28 Room Z Pen D Litter 17 con 5.928738
## 29 Room Z Pen D Litter 18 vac 7.622600
## 30 Room Z Pen D Litter 18 vac 5.036121
## 31 Room Z Pen D Litter 18 con 8.620428
## 32 Room Z Pen D Litter 18 con 6.265424
## 33 Room Z Pen E Litter 19 vac 3.787388
## 34 Room Z Pen E Litter 19 vac 5.380694
## 35 Room Z Pen E Litter 19 con 9.454640
## 36 Room Z Pen E Litter 19 con 6.505600
## 37 Room Z Pen E Litter 20 vac 5.300123
## 38 Room Z Pen E Litter 20 vac 4.417709
## 39 Room Z Pen E Litter 20 con 7.211453
## 40 Room Z Pen E Litter 20 con 6.351508
## 41 Room Z Pen F Litter 21 vac 4.690320
## 42 Room Z Pen F Litter 21 vac 6.035818
## 43 Room Z Pen F Litter 21 con 6.643916
## 44 Room Z Pen F Litter 21 con 6.995050
## 45 Room Z Pen F Litter 22 vac 4.896764
## 46 Room Z Pen F Litter 22 vac 5.371713
## 47 Room Z Pen F Litter 22 con 6.773614
```

1 Quick Start

The two functions `MFClusHier` and `MFClusBootHier` are wrapper functions that allow calculation of MF and bootstrapped MF, respectively, as a single step.

Calculated MF.

`MFClusHier` requires four inputs:

- `formula`: The formula in form `y ~ x + a/b/c` where `y` is a continuous response, `x` is a factor with two levels of treatment, and `a/b/c` are variables corresponding to the clusters. It is expected that levels of "c" are nested within levels of "b". Nesting is assumed to be in order, left to right, highest to lowest.
- `data`: The data table (`data.frame` or `tibble`) with variables as identified in formula. If there are extra variables, they will be ignored.

- `compare`: A text vector of length two. The first position [1] is treated as the control or reference group two which members of the second position [2] are compared. If no value for the compare vector is supplied, it is assumed that the comparisons are between the `con` and `vac` treatment groups.
- `which.factor`: One or more variable(s) of interest. This can be any of the core or nest variables from the data set. If none or NULL is specified, MF will be calculated for the whole tree.

Common Usage

Default case, looking only at whole tree:

```
mf_all <- MFClusHier(formula = lung ~ tx + room/pen/litter, data = a)
```

Selecting a specific nest variable:

```
mf_room <- MFClusHier(formula = lung ~ tx + room/pen/litter, data = a,
                      which.factor = 'room')
```

Selecting multiple variables, including the entire tree:

```
mf_multiple <- MFClusHier(formula = lung ~ tx + room/pen/litter, data = a,
                      which.factor = c('room', 'litter', 'All'))
```

Output

Output object is a list of `MFh` and `MFnest`. The first item is a rank table that can be used to reproduce MF calculations or update selection of choice for `which.factor`. The second item is a table that shows the MF for each unique level of the variable(s) specified in `which.factor`.

```
mf_all
```

```
## $MFh
## # A tibble: 12 x 10
##    room   pen   litter    con_medResp con_n     w vac_medResp vac_n  n1n2
##    <chr>  <chr> <chr>           <dbl> <dbl> <dbl>       <dbl> <dbl> <dbl>
##  1 Room W Pen A Litter 11        8.24     2     7        5.13     2     4
##  2 Room W Pen A Litter 12        4.91     2     5        3.81     2     4
##  3 Room W Pen B Litter 13        8.10     2     7        5.23     2     4
##  4 Room W Pen B Litter 14        8.11     2     7        5.59     2     4
##  5 Room W Pen C Litter 15        8.09     2     7        5.26     2     4
##  6 Room W Pen C Litter 16        6.77     2     7        4.50     2     4
##  7 Room Z Pen D Litter 17        5.58     2     7        4.26     2     4
##  8 Room Z Pen D Litter 18        7.44     2     6        6.33     2     4
##  9 Room Z Pen E Litter 19        7.98     2     7        4.58     2     4
## 10 Room Z Pen E Litter 20        6.78     2     7        4.86     2     4
## 11 Room Z Pen F Litter 21        6.82     2     7        5.36     2     4
## 12 Room Z Pen F Litter 22        6.77     1     3        5.13     2     2
## # ... with 1 more variable: u <dbl>
```

```
##
## $MFnest
## # A tibble: 1 x 9
##   variable level    MF  N1N2     U con_N vac_N con_medResp vac_medResp
##   <fct>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl>       <dbl>       <dbl>
## 1 All      All   0.870    46    43    23    24        7.21        4.91
```

Within the `MFh` table the variables are:

- One row per unique core variable. The input data structure will determine how many variables and rows it takes to fully enumerate this.
- `con_n` & `vac_n`: Counts of observations for each treatment level in the particular instance of core or nest variable. Note that the left hand side of the underscore will match values passed by user to `compare`.
- `n1n2`: Product of counts, `con_n` * `vac_n`.
- `w`: Wilcoxon statistic.
- `u`: Mann-Whitney statistic.
- `con_medResp` & `vac_medResp`: Median observation for for each treatment level in the particular instance of core or nest variable. Note that the left hand side of the underscore will match values passed by user to `compare`.

Within the `MFnest` table, the variables are:

- `variable`: Which core or nest variable is of interest for the row.
- `level`: A unique case of the variable. Input data determines how many rows it takes to enumerate a particular variable.
- `MF`: Mitigated fraction calculated for the particular level.
- `N1N2`: Sum of the `n1n2` values from table `MFh` matching the particular level of interest in the row.
- `U`: Sum of the `u` values from table `MFh` matching the particular level of interest in the row.
- `con_N` & `vac_N`: Sum of the `_n` values from table `MFh` matching the particular level of interest in the row. Note that the left hand side of the underscore will match values passed by user to `compare`.
- `con_medResp` & `vac_medResp`: Median of responses for each comparison group matching the particular level of interest in the row. Note that the left hand side of the underscore will match values passed by user to `compare`.

Bootstrapped MF.

`MFClusBootHier` uses all of the inputs as above plus the additional four values:

- `nboot`: Number of bootstrapping events.
- `boot.unit`: Boolean whether to sample observations from within those of the same core.

- `boot.cluster`: Boolean whether to sample which cores are present. If TRUE, some trees have all the cores represented in the original data while others only have a subset.
- `alpha`: Used to calculate empirical hpd by shortest length interval.

Common Usage

Default case, looking only at whole tree and bootstrapping both at the cluster and unit levels:

```
mfboot_all <- MFClusBootHier(formula = lung ~ tx + room/pen/litter,
                             data = a)
```

Specifying what bootstrapping sampling to occur:

```
mfboot_room <- MFClusBootHier(formula = lung ~ tx + room/pen/litter,
                              data = a, boot.unit = TRUE,
                              boot.cluster = FALSE, which.factor = 'room')
```

Adjusting the number of bootstrapping events and alpha:

```
mfboot_multiple <- MFClusBootHier(formula = lung ~ tx + room/pen/litter,
                                  data = a, boot.unit = FALSE,
                                  boot.cluster = TRUE,  alpha = 0.1,
                                  which.factor = c('room', 'litter', 'All'))
```

Output

Output object is a list of `MFhBoot` and `MFnestBoot`. The former is a list of elements that includes:

- `clusters`: The unique clusters from the original data.
- `bootmfh`: A rank table for all the bootstrapping events. For each bootstrapping event, there are the number of clusters equivalent to the number of unique clusters in the original data. If boot.cluster == TRUE, a random sampling with replacement will be represented in each bootstrapping event. Each cluster retains characteristics from the original cluster, such as number of observations and the calculated statistics, which will be pooled when calculating the MF.
- `compare`: The values passed to compare by user.
- `mfh`: The rank table calculated from the original data.

The latter is a list of tables including:

- `mfnest_summary`: The MF for each unique level of the variable(s) specified in `which.factor`, including summary statistics of distribution observed by bootstrapping.
- `mfnest_details`: The MF for each bootstrapping event for the variable(s) specified in `which.factor`.

```
mfboot_all
```

```
## $MFhBoot
## $MFhBoot$bootmfh
## # A tibble: 120,000 x 11
```

```
##    bootID     w     u  n1n2 con_n vac_n con_medResp vac_medResp room
##     <int> <dbl> <dbl> <int> <int> <int>       <dbl>       <dbl> <chr>
##  1      1     7     4     4     2     2        6.78        4.86 Room Z
##  2      1     7     4     4     2     2        8.24        5.13 Room Z
##  3      1     7     4     4     2     2        7.98        4.58 Room Z
##  4      1     7     4     4     2     2        5.58        4.26 Room Z
##  5      1     7     4     4     2     2        8.11        5.59 Room W
##  6      1     7     4     4     2     2        8.11        5.59 Room W
##  7      1     3     0     4     2     2        4.91        3.81 Room Z
##  8      1     3     0     4     2     2        4.91        3.81 Room W
##  9      1     7     4     4     2     2        6.77        4.50 Room Z
## 10      1     3     2     2     1     2        6.77        5.13 Room W
## # ... with 119,990 more rows, and 2 more variables: pen <chr>,
## #   litter <chr>
##
## $MFhBoot$clusters
##      room   pen    litter clusterID
## 1  Room W Pen A Litter 11         1
## 2  Room W Pen A Litter 12         2
## 3  Room W Pen B Litter 13         3
## 4  Room W Pen B Litter 14         4
## 5  Room W Pen C Litter 15         5
## 6  Room W Pen C Litter 16         6
## 7  Room Z Pen D Litter 17         7
## 8  Room Z Pen D Litter 18         8
## 9  Room Z Pen E Litter 19         9
## 10 Room Z Pen E Litter 20        10
## 11 Room Z Pen F Litter 21        11
## 12 Room Z Pen F Litter 22        12
##
## $MFhBoot$compare
## [1] "con" "vac"
##
## $MFhBoot$mfh
## # A tibble: 12 x 10
##    room   pen   litter    con_medResp con_n     w vac_medResp vac_n  n1n2
##    <chr>  <chr> <chr>           <dbl> <dbl> <dbl>       <dbl> <dbl> <dbl>
##  1 Room W Pen A Litter 11        8.24     2     7        5.13     2     4
##  2 Room W Pen A Litter 12        4.91     2     5        3.81     2     4
##  3 Room W Pen B Litter 13        8.10     2     7        5.23     2     4
##  4 Room W Pen B Litter 14        8.11     2     7        5.59     2     4
##  5 Room W Pen C Litter 15        8.09     2     7        5.26     2     4
##  6 Room W Pen C Litter 16        6.77     2     7        4.50     2     4
##  7 Room Z Pen D Litter 17        5.58     2     7        4.26     2     4
##  8 Room Z Pen D Litter 18        7.44     2     6        6.33     2     4
```

```
##  9 Room Z Pen E Litter 19        7.98     2     7        4.58     2     4
## 10 Room Z Pen E Litter 20        6.78     2     7        4.86     2     4
## 11 Room Z Pen F Litter 21        6.82     2     7        5.36     2     4
## 12 Room Z Pen F Litter 22        6.77     1     3        5.13     2     2
## # ... with 1 more variable: u <dbl>
##
##
## $MFnestBoot
## $MFnestBoot$mfnest_details
## # A tibble: 10,000 x 8
## # Groups:   variable, level [?]
##    variable level bootID     U  N1N2 con_N vac_N    MF
##    <chr>    <chr>  <int> <dbl> <int> <int> <int> <dbl>
##  1 All      All        1    36    44    22    24 0.636
##  2 All      All        2    42    44    22    24 0.909
##  3 All      All        3    41    46    23    24 0.783
##  4 All      All        4    43    46    23    24 0.870
##  5 All      All        5    42    44    22    24 0.909
##  6 All      All        6    37    46    23    24 0.609
##  7 All      All        7    42    46    23    24 0.826
##  8 All      All        8    44    46    23    24 0.913
##  9 All      All        9    44    44    22    24 1
## 10 All      All       10    39    44    22    24 0.773
## # ... with 9,990 more rows
##
## $MFnestBoot$mfnest_summary
## # A tibble: 1 x 8
##   variable level median etlower etupper htlower htupper mf.obs
##   <fct>    <chr>  <dbl>   <dbl>   <dbl>   <dbl>   <dbl>  <dbl>
## 1 All      All    0.909   0.583       1   0.652       1  0.870
```

2 Reproducibility from rank tables

It is possible to refer to the rank table to reproduce calculations of MF or to update the choice of `which.factor`.

Calculated MF.

The function `MFnest` allows us to calculate the MF of designated nest variables given a rank table. If `MFClusHier` was previously called on the data, this information is captured in the `MFh` value:

```r
mf_all$MFh
```

```
## # A tibble: 12 x 10
##    room   pen   litter    con_medResp con_n     w vac_medResp vac_n  n1n2
##    <chr>  <chr> <chr>           <dbl> <dbl> <dbl>       <dbl> <dbl> <dbl>
##  1 Room W Pen A Litter 11        8.24     2     7        5.13     2     4
##  2 Room W Pen A Litter 12        4.91     2     5        3.81     2     4
##  3 Room W Pen B Litter 13        8.10     2     7        5.23     2     4
##  4 Room W Pen B Litter 14        8.11     2     7        5.59     2     4
##  5 Room W Pen C Litter 15        8.09     2     7        5.26     2     4
##  6 Room W Pen C Litter 16        6.77     2     7        4.50     2     4
##  7 Room Z Pen D Litter 17        5.58     2     7        4.26     2     4
##  8 Room Z Pen D Litter 18        7.44     2     6        6.33     2     4
##  9 Room Z Pen E Litter 19        7.98     2     7        4.58     2     4
## 10 Room Z Pen E Litter 20        6.78     2     7        4.86     2     4
## 11 Room Z Pen F Litter 21        6.82     2     7        5.36     2     4
## 12 Room Z Pen F Litter 22        6.77     1     3        5.13     2     2
## # ... with 1 more variable: u <dbl>
```

The only other value needed is the selection passed to `which.factor`.

Common Usage

By default, MF for the whole tree is calculated:

```r
MFnest(mf_all$MFh)
```

```
## # A tibble: 1 x 9
##   variable level    MF  N1N2     U con_N vac_N con_medResp vac_medResp
##   <fct>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl>       <dbl>       <dbl>
## 1 All      All   0.870    46    43    23    24        7.21        4.91
```

But the user can specify one or more variables for which to evaluate MF.

```r
MFnest(mf_all$MFh, which.factor = 'room')
```

```
## # A tibble: 2 x 9
##   variable level     MF  N1N2     U con_N vac_N con_medResp vac_medResp
##   <fct>    <chr>  <dbl> <dbl> <dbl> <dbl> <dbl>       <dbl>       <dbl>
## 1 room     Room W 0.833    24    22    12    12        7.79        4.85
## 2 room     Room Z 0.909    22    21    11    12        6.64        4.98
```

```r
MFnest(mf_all$MFh, which.factor = c('room', 'litter', 'All'))
```

```
## Complete separation observed for variable(s): litter
```

```
## # A tibble: 15 x 9
##    variable level       MF  N1N2     U con_N vac_N con_medResp vac_medResp
##    <fct>    <chr>    <dbl> <dbl> <dbl> <dbl> <dbl>       <dbl>       <dbl>
```

```
##  1 room     Room W   0.833    24    22    12    12        7.79        4.85
##  2 room     Room Z   0.909    22    21    11    12        6.64        4.98
##  3 litter   Litter ~ 1         4     4     2     2        8.24        5.13
##  4 litter   Litter ~ 0         4     2     2     2        4.91        3.81
##  5 litter   Litter ~ 1         4     4     2     2        8.10        5.23
##  6 litter   Litter ~ 1         4     4     2     2        8.11        5.59
##  7 litter   Litter ~ 1         4     4     2     2        8.09        5.26
##  8 litter   Litter ~ 1         4     4     2     2        6.77        4.50
##  9 litter   Litter ~ 1         4     4     2     2        5.58        4.26
## 10 litter   Litter ~ 0.5       4     3     2     2        7.44        6.33
## 11 litter   Litter ~ 1         4     4     2     2        7.98        4.58
## 12 litter   Litter ~ 1         4     4     2     2        6.78        4.86
## 13 litter   Litter ~ 1         4     4     2     2        6.82        5.36
## 14 litter   Litter ~ 1         2     2     1     2        6.77        5.13
## 15 All      All      0.870    46    43    23    24        7.21        4.91
```

Bootstrapped MF.

The function `MFnestBoot` allows to calculate the MF of designated nest variables given rank tables from bootstrapped and original data. If `MFClusBootHier` was previously called on the data, this information is captured in the `MFhBoot` value:

```
mfboot_all$MFhBoot
```

```
## $bootmfh
## # A tibble: 120,000 x 11
##    bootID     w     u  n1n2 con_n vac_n con_medResp vac_medResp room
##     <int> <dbl> <dbl> <int> <int> <int>       <dbl>       <dbl> <chr>
##  1      1     7     4     4     2     2        6.78        4.86 Room Z
##  2      1     7     4     4     2     2        8.24        5.13 Room Z
##  3      1     7     4     4     2     2        7.98        4.58 Room Z
##  4      1     7     4     4     2     2        5.58        4.26 Room Z
##  5      1     7     4     4     2     2        8.11        5.59 Room W
##  6      1     7     4     4     2     2        8.11        5.59 Room W
##  7      1     3     0     4     2     2        4.91        3.81 Room Z
##  8      1     3     0     4     2     2        4.91        3.81 Room W
##  9      1     7     4     4     2     2        6.77        4.50 Room Z
## 10      1     3     2     2     1     2        6.77        5.13 Room W
## # ... with 119,990 more rows, and 2 more variables: pen <chr>,
## #   litter <chr>
##
## $clusters
##      room   pen    litter clusterID
## 1  Room W Pen A Litter 11         1
```

```
## 2  Room W Pen A Litter 12         2
## 3  Room W Pen B Litter 13         3
## 4  Room W Pen B Litter 14         4
## 5  Room W Pen C Litter 15         5
## 6  Room W Pen C Litter 16         6
## 7  Room Z Pen D Litter 17         7
## 8  Room Z Pen D Litter 18         8
## 9  Room Z Pen E Litter 19         9
## 10 Room Z Pen E Litter 20        10
## 11 Room Z Pen F Litter 21        11
## 12 Room Z Pen F Litter 22        12
##
## $compare
## [1] "con" "vac"
##
## $mfh
## # A tibble: 12 x 10
##    room   pen   litter    con_medResp con_n     w vac_medResp vac_n  n1n2
##    <chr>  <chr> <chr>           <dbl> <dbl> <dbl>       <dbl> <dbl> <dbl>
##  1 Room W Pen A Litter 11        8.24     2     7        5.13     2     4
##  2 Room W Pen A Litter 12        4.91     2     5        3.81     2     4
##  3 Room W Pen B Litter 13        8.10     2     7        5.23     2     4
##  4 Room W Pen B Litter 14        8.11     2     7        5.59     2     4
##  5 Room W Pen C Litter 15        8.09     2     7        5.26     2     4
##  6 Room W Pen C Litter 16        6.77     2     7        4.50     2     4
##  7 Room Z Pen D Litter 17        5.58     2     7        4.26     2     4
##  8 Room Z Pen D Litter 18        7.44     2     6        6.33     2     4
##  9 Room Z Pen E Litter 19        7.98     2     7        4.58     2     4
## 10 Room Z Pen E Litter 20        6.78     2     7        4.86     2     4
## 11 Room Z Pen F Litter 21        6.82     2     7        5.36     2     4
## 12 Room Z Pen F Litter 22        6.77     1     3        5.13     2     2
## # ... with 1 more variable: u <dbl>
```

The other values needed are the selection passed to `which.factor` and `alpha`.

Common Usage

By default, MF for the whole tree is calculated using an *alpha = 0.05*:

```r
MFnestBoot(mfboot_all$MFhBoot)
```

```
## $mfnest_details
## # A tibble: 10,000 x 8
## # Groups:   variable, level [?]
##    variable level bootID     U  N1N2 con_N vac_N    MF
##    <chr>    <chr>  <int> <dbl> <int> <int> <int> <dbl>
##  1 All      All        1    36    44    22    24 0.636
```

```
##  2 All      All        2    42    44    22    24 0.909
##  3 All      All        3    41    46    23    24 0.783
##  4 All      All        4    43    46    23    24 0.870
##  5 All      All        5    42    44    22    24 0.909
##  6 All      All        6    37    46    23    24 0.609
##  7 All      All        7    42    46    23    24 0.826
##  8 All      All        8    44    46    23    24 0.913
##  9 All      All        9    44    44    22    24 1
## 10 All      All       10    39    44    22    24 0.773
## # ... with 9,990 more rows
##
## $mfnest_summary
## # A tibble: 1 x 8
##   variable level median etlower etupper htlower htupper mf.obs
##   <fct>    <chr>  <dbl>   <dbl>   <dbl>   <dbl>   <dbl>  <dbl>
## 1 All      All    0.909   0.583       1   0.652       1  0.870
```

But the user can specify one or more variables for which to evaluate MF.

```r
MFnestBoot(mfboot_all$MFh, which.factor = 'room')
```

```
## $mfnest_details
## # A tibble: 19,996 x 8
## # Groups:   variable, level [?]
##    variable level  bootID     U  N1N2 con_N vac_N    MF
##    <chr>    <chr>   <int> <dbl> <int> <int> <int> <dbl>
##  1 room     Room W      1    16    20    10    12 0.6
##  2 room     Room W      2    34    36    18    20 0.889
##  3 room     Room W      3     7    12     6     6 0.167
##  4 room     Room W      4    18    20    10    10 0.8
##  5 room     Room W      5    10    10     5     6 1
##  6 room     Room W      6    21    26    13    14 0.615
##  7 room     Room W      7     8     8     4     4 1
##  8 room     Room W      8    20    20    10    10 1
##  9 room     Room W      9    30    30    15    16 1
## 10 room     Room W     10    23    24    12    12 0.917
## # ... with 19,986 more rows
##
## $mfnest_summary
## # A tibble: 2 x 8
##   variable level  median etlower etupper htlower htupper mf.obs
##   <fct>    <chr>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>  <dbl>
## 1 room     Room W      1   0.411       1     0.5       1  0.833
## 2 room     Room Z      1   0.400       1     0.5       1  0.909
```

```r
MFnestBoot(mfboot_all$MFh, which.factor = c('room', 'litter', 'All'),
           alpha = 0.1)
```

```
## Complete separation observed for variable(s): litter
```

```
## $mfnest_details
## # A tibble: 107,872 x 8
## # Groups:   variable, level [?]
##    variable level bootID     U  N1N2 con_N vac_N    MF
##    <chr>    <chr>  <int> <dbl> <int> <int> <int> <dbl>
##  1 All      All        1    36    44    22    24 0.636
##  2 All      All        2    42    44    22    24 0.909
##  3 All      All        3    41    46    23    24 0.783
##  4 All      All        4    43    46    23    24 0.870
##  5 All      All        5    42    44    22    24 0.909
##  6 All      All        6    37    46    23    24 0.609
##  7 All      All        7    42    46    23    24 0.826
##  8 All      All        8    44    46    23    24 0.913
##  9 All      All        9    44    44    22    24 1
## 10 All      All       10    39    44    22    24 0.773
## # ... with 107,862 more rows
##
## $mfnest_summary
## # A tibble: 15 x 8
##    variable level     median etlower etupper htlower htupper mf.obs
##    <fct>    <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>  <dbl>
##  1 room     Room W     1       0.5         1   0.625       1  0.833
##  2 room     Room Z     1       0.5         1   0.625       1  0.909
##  3 litter   Litter 11  1       0           1   0.5         1  1
##  4 litter   Litter 12  1       0           1   0.5         1  0
##  5 litter   Litter 13  1       0           1   0.5         1  1
##  6 litter   Litter 14  1       0           1   0.5         1  1
##  7 litter   Litter 15  1       0           1   0.5         1  1
##  8 litter   Litter 16  1       0           1   0.5         1  1
##  9 litter   Litter 17  1       0           1   0.5         1  1
## 10 litter   Litter 18  1       0           1   0.5         1  0.5
## 11 litter   Litter 19  1       0           1   0.5         1  1
## 12 litter   Litter 20  1       0           1   0.5         1  1
## 13 litter   Litter 21  1       0           1   0.5         1  1
## 14 litter   Litter 22  1       0           1   0.5         1  1
## 15 All      All        0.909   0.652       1   0.708       1  0.870
```

Note that there is no sampling performed in this step. All sampling is performed during calculation of the rank tables.

3 Calculating the rank table directly

It is also possible to calculate the rank tables independently. This might be useful if the rank table is needed for other calculations separate from the activity of calculating MF.

Calculated

The function `MFh` can be used to calculate the rank table. The usage is very similar to the wrapper function:

```
MFh(formula = lung ~ tx + room/pen/litter, data = a)
```

```
## # A tibble: 12 x 10
##    room   pen   litter    con_medResp con_n     w vac_medResp vac_n  n1n2
##    <chr>  <chr> <chr>           <dbl> <dbl> <dbl>       <dbl> <dbl> <dbl>
##  1 Room W Pen A Litter 11        8.24     2     7        5.13     2     4
##  2 Room W Pen A Litter 12        4.91     2     5        3.81     2     4
##  3 Room W Pen B Litter 13        8.10     2     7        5.23     2     4
##  4 Room W Pen B Litter 14        8.11     2     7        5.59     2     4
##  5 Room W Pen C Litter 15        8.09     2     7        5.26     2     4
##  6 Room W Pen C Litter 16        6.77     2     7        4.50     2     4
##  7 Room Z Pen D Litter 17        5.58     2     7        4.26     2     4
##  8 Room Z Pen D Litter 18        7.44     2     6        6.33     2     4
##  9 Room Z Pen E Litter 19        7.98     2     7        4.58     2     4
## 10 Room Z Pen E Litter 20        6.78     2     7        4.86     2     4
## 11 Room Z Pen F Litter 21        6.82     2     7        5.36     2     4
## 12 Room Z Pen F Litter 22        6.77     1     3        5.13     2     2
## # ... with 1 more variable: u <dbl>
```

To manually calculate the MF without use of wrapper function, string together calls to both `MFh` and `MFnest`:

```
thismf <- MFh(formula = lung ~ tx + room/pen/litter, data = a)
MFnest(thismf)
```

```
## # A tibble: 1 x 9
##   variable level    MF  N1N2     U con_N vac_N con_medResp vac_medResp
##   <fct>    <chr> <dbl> <dbl> <dbl> <dbl> <dbl>       <dbl>       <dbl>
## 1 All      All   0.870    46    43    23    24        7.21        4.91
```

Bootstrapped

The function `MFhBoot` can be used to calculate bootstrapped rank table. Usage is similar to the wrapper function:

```r
MFhBoot(formula = lung ~ tx + room/pen/litter, data = a, nboot = 10000,
  boot.unit = TRUE, boot.cluster = TRUE)
```

```
## $bootmfh
## # A tibble: 120,000 x 11
##    bootID     w     u  n1n2 con_n vac_n con_medResp vac_medResp room
##     <int> <dbl> <dbl> <int> <int> <int>       <dbl>       <dbl> <chr>
##  1      1     7     4     4     2     2        8.11        5.59 Room W
##  2      1     7     4     4     2     2        7.44        6.33 Room W
##  3      1     6     3     4     2     2        7.44        6.33 Room W
##  4      1     7     4     4     2     2        8.24        5.13 Room W
##  5      1     7     4     4     2     2        6.82        5.36 Room Z
##  6      1     7     4     4     2     2        8.24        5.13 Room W
##  7      1     7     4     4     2     2        8.11        5.59 Room W
##  8      1     7     4     4     2     2        5.58        4.26 Room Z
##  9      1     7     4     4     2     2        6.82        5.36 Room W
## 10      1     3     2     2     1     2        6.77        5.13 Room Z
## # ... with 119,990 more rows, and 2 more variables: pen <chr>,
## #   litter <chr>
##
## $clusters
##      room   pen    litter clusterID
## 1  Room W Pen A Litter 11         1
## 2  Room W Pen A Litter 12         2
## 3  Room W Pen B Litter 13         3
## 4  Room W Pen B Litter 14         4
## 5  Room W Pen C Litter 15         5
## 6  Room W Pen C Litter 16         6
## 7  Room Z Pen D Litter 17         7
## 8  Room Z Pen D Litter 18         8
## 9  Room Z Pen E Litter 19         9
## 10 Room Z Pen E Litter 20        10
## 11 Room Z Pen F Litter 21        11
## 12 Room Z Pen F Litter 22        12
##
## $compare
## [1] "con" "vac"
##
## $mfh
## # A tibble: 12 x 10
##    room   pen   litter    con_medResp con_n     w vac_medResp vac_n  n1n2
##    <chr>  <chr> <chr>           <dbl> <dbl> <dbl>       <dbl> <dbl> <dbl>
##  1 Room W Pen A Litter 11        8.24     2     7        5.13     2     4
##  2 Room W Pen A Litter 12        4.91     2     5        3.81     2     4
```

```
##  3 Room W Pen B Litter 13        8.10     2     7        5.23     2     4
##  4 Room W Pen B Litter 14        8.11     2     7        5.59     2     4
##  5 Room W Pen C Litter 15        8.09     2     7        5.26     2     4
##  6 Room W Pen C Litter 16        6.77     2     7        4.50     2     4
##  7 Room Z Pen D Litter 17        5.58     2     7        4.26     2     4
##  8 Room Z Pen D Litter 18        7.44     2     6        6.33     2     4
##  9 Room Z Pen E Litter 19        7.98     2     7        4.58     2     4
## 10 Room Z Pen E Litter 20        6.78     2     7        4.86     2     4
## 11 Room Z Pen F Litter 21        6.82     2     7        5.36     2     4
## 12 Room Z Pen F Litter 22        6.77     1     3        5.13     2     2
## # ... with 1 more variable: u <dbl>
```

To manually calculate the MF without use of wrapper function, string together calls to both `MFhBoot` and `MFnestBoot`:

```r
thismfboot <- MFhBoot(formula = lung ~ tx + room/pen/litter, data = a)
MFnestBoot(thismfboot)
```

```
## $mfnest_details
## # A tibble: 10,000 x 8
## # Groups:   variable, level [?]
##    variable level bootID     U  N1N2 con_N vac_N    MF
##    <chr>    <chr>  <int> <dbl> <int> <int> <int> <dbl>
##  1 All      All        1    42    44    22    24 0.909
##  2 All      All        2    38    46    23    24 0.652
##  3 All      All        3    45    46    23    24 0.957
##  4 All      All        4    42    44    22    24 0.909
##  5 All      All        5    48    48    24    24 1
##  6 All      All        6    44    46    23    24 0.913
##  7 All      All        7    46    48    24    24 0.917
##  8 All      All        8    38    46    23    24 0.652
##  9 All      All        9    41    48    24    24 0.708
## 10 All      All       10    44    46    23    24 0.913
## # ... with 9,990 more rows
##
## $mfnest_summary
## # A tibble: 1 x 8
##   variable level median etlower etupper htlower htupper mf.obs
##   <fct>    <chr>  <dbl>   <dbl>   <dbl>   <dbl>   <dbl>  <dbl>
## 1 All      All    0.909   0.583       1   0.652       1  0.870
```

Appendix I

room	pen	litter	tx	lung
Room W	Pen A	Litter 11	vac	5.633023
				4.618143
			con	9.196908
				7.277479
		Litter 12	vac	3.760431
				3.856939
			con	6.307358
				3.521230
	Pen B	Litter 13	vac	4.356031
				6.098324
			con	7.866934
				8.339948
		Litter 14	vac	5.389661
				5.799638
			con	8.275256
				7.952039
	Pen C	Litter 15	vac	4.855498
				5.657725
			con	7.658824
				8.517004
		Litter 16	vac	4.169034
				4.837650
			con	5.822156
				7.718757
Room Z	Pen D	Litter 17	vac	3.595465
				4.921762
			con	5.222559
				5.928738
		Litter 18	vac	7.622600
				5.036120
			con	8.620428
				6.265423
	Pen E	Litter 19	vac	3.787388
				5.380694
			con	9.454640
				6.505599
		Litter 20	vac	5.300123
				4.417709
			con	7.211453
				6.351508
	Pen F	Litter 21	vac	4.690320
				6.035818
			con	6.643916
				6.995050
		Litter 22	vac	4.896763
				5.371713
			con	6.773614

Appendix II

```r
sessionInfo()
```

```
## R version 3.4.4 (2018-03-15)
## Platform: x86_64-w64-mingw32/x64 (64-bit)
```

```
## Running under: Windows 10 x64 (build 14393)
##
## Matrix products: default
##
## locale:
## [1] LC_COLLATE=English_United States.1252
## [2] LC_CTYPE=English_United States.1252
## [3] LC_MONETARY=English_United States.1252
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.1252
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] bindrcpp_0.2.2   MF_4.3.5         forcats_0.3.0    stringr_1.3.1
##  [5] dplyr_0.7.7      purrr_0.2.5      readr_1.1.1      tidyr_0.8.1
##  [9] tibble_1.4.2     ggplot2_3.0.0    tidyverse_1.2.1  kableExtra_0.9.0
##
## loaded via a namespace (and not attached):
##  [1] Rcpp_0.12.17      cellranger_1.1.0  pillar_1.2.3
##  [4] compiler_3.4.4    plyr_1.8.4        bindr_0.1.1
##  [7] tools_3.4.4       digest_0.6.15     lubridate_1.7.4
## [10] jsonlite_1.5      gtable_0.2.0      evaluate_0.10.1
## [13] nlme_3.1-131.1    viridisLite_0.3.0 lattice_0.20-35
## [16] pkgconfig_2.0.1   rlang_0.2.1       cli_1.0.0
## [19] rstudioapi_0.7    yaml_2.1.19       haven_1.1.2
## [22] withr_2.1.2       httr_1.3.1        xml2_1.2.0
## [25] knitr_1.20        hms_0.4.2         grid_3.4.4
## [28] rprojroot_1.3-2   tidyselect_0.2.4  glue_1.2.0
## [31] R6_2.2.2          readxl_1.1.0      rmarkdown_1.10
## [34] modelr_0.1.2      magrittr_1.5      backports_1.1.2
## [37] scales_0.5.0      htmltools_0.3.6   assertthat_0.2.0
## [40] rvest_0.3.2       colorspace_1.3-2  utf8_1.1.4
## [43] stringi_1.1.7     lazyeval_0.2.1    munsell_0.5.0
## [46] broom_0.5.0       crayon_1.3.4
```